Contents

SYSTEM DESIGN MASTERY ROADMAP **1**
From Fundamentals to Architecture: Ace Your System Design Interviews 1
1. THE SYSTEM DESIGN LANDSCAPE 1
2. PHASE 0: COMPUTER SCIENCE REFRESHER 2
3. PHASE 1: FUNDAMENTALS & CORE CONCEPTS 3
4. PHASE 2: STORAGE & DATABASES 8
5. PHASE 3: COMMUNICATION & APIS 13
6. PHASE 4: DISTRIBUTED SYSTEMS 18
7. PHASE 5: HIGH-LEVEL DESIGN CASE STUDIES 23
8. PHASE 6: LOW-LEVEL DESIGN & OOP 31
9. PHASE 7: INTERVIEW PREPARATION 37
10. TOOLS & TECHNOLOGIES TABLE 39
11. RESOURCES & BOOKS 41
FINAL WORDS 43

SYSTEM DESIGN MASTERY ROADMAP

From Fundamentals to Architecture: Ace Your System Design Interviews

Current Status (June 2026): - Language: C++ - Experience: Entry level - Goal: FAANG+ SDE/SWE roles - Phase: Learning System Design

---

1. THE SYSTEM DESIGN LANDSCAPE

What is System Design?

System Design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It is the bridge between requirements and implementation.

Why System Design Matters

```
- Senior engineers spend 60%+ time on design decisions
- FAANG interviews: 3-5 rounds of system design
- Bad design = technical debt, scaling nightmares
- Good design = maintainable, scalable, reliable systems

Common Interview Formats:
  - High-Level Design (HLD): "Design Twitter"
  - Low-Level Design (LLD): "Design a parking lot system"
  - Object-Oriented Design: "Design a chess game"
```

System Design Interview Assessment Criteria

Criterion	Weight	What They Look For
Requirements Gathering	15%	Asks clarifying questions, defines scope
Scalability Thinking	25%	Handles traffic growth, sharding, caching
Data Modeling	20%	Database choice, schema design
API Design	10%	REST/gRPC, endpoints, error handling

Criterion	Weight	What They Look For
Tradeoff Analysis	20%	Compares options, justifies decisions
Communication	10%	Clear explanations, diagrams, structured approach

---

2. PHASE 0: COMPUTER SCIENCE REFRESHER

Duration: 1-2 weeks

Networking Basics for System Design

```
OSI Model:
  7. Application: HTTP, FTP, SMTP
  6. Presentation: SSL/TLS, encryption
  5. Session: NetBIOS, RPC
  4. Transport: TCP, UDP
  3. Network: IP, routing
  2. Data Link: Ethernet, MAC
  1. Physical: cables, signals

TCP vs UDP:
  TCP: connection-oriented, reliable, ordered, congestion control
    - 3-way handshake: SYN → SYN-ACK → ACK
    - Flow control: sliding window
    - Congestion: AIMD, slow start, fast recovery
  UDP: connectionless, unreliable, no ordering
    - Low latency, no retransmission
    - Use: video streaming, DNS, VoIP

HTTP Protocol:
  HTTP/1.1: persistent connections, pipelining (limited)
    - Head-of-line blocking at application layer
  HTTP/2: multiplexed streams, binary framing, header compression (HPACK)
    - Single TCP connection, server push
    - Head-of-line blocking at transport layer
  HTTP/3: QUIC (UDP-based), 0-RTT handshake
    - Eliminates TCP HOL blocking
    - Built-in encryption (TLS 1.3)
    - Connection migration (works across network changes)

  Common Status Codes:
    200 OK, 201 Created, 204 No Content
    301 Moved Permanently, 302 Found
    400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
    429 Too Many Requests
    500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable
```

Operating Systems Refresher

```
Process vs Thread:
  Process: independent execution unit, separate address space
    - Fork, exec, wait system calls
    - Inter-process communication: pipes, sockets, shared memory, message queues
  Thread: light-weight, shares address space with process
    - User-level vs Kernel-level threads
    - Context switching overhead: threads < processes
```

```
Concurrency:
  Synchronization primitives:
    - Mutex: mutual exclusion
    - Semaphore: counting semaphore (producer-consumer)
    - Read-Write Lock: multiple readers, single writer
    - Condition Variable: wait/notify pattern
    - Barrier: wait for all threads to reach point

  Deadlock: 4 conditions (mutual exclusion, hold & wait, no preemption, circular wait)
  Starvation: thread never gets resources
  Race condition: outcome depends on timing

Scheduling:
  - FIFO, SJF, Round Robin, Priority, Multi-level Feedback Queue
  - CPU-bound vs I/O-bound processes

Memory Management:
  - Virtual memory: MMU, page tables, TLB
  - Paging: fixed-size pages, page faults
  - Segmentation: variable-size segments
  - Thrashing: excessive paging
  - Cache hierarchy: L1 (per-core), L2 (per-core), L3 (shared)
```

---

3. PHASE 1: FUNDAMENTALS & CORE CONCEPTS

Duration: 3-4 weeks

3.1 CAP Theorem

```
CAP Theorem (Brewer's Theorem):
  In a distributed system, you can have at most 2 of 3 guarantees:

  Consistency (C): Every read receives the most recent write or error
    - After a write, all nodes see the same data

  Availability (A): Every request receives a response (not necessarily latest data)
    - System remains operational even if nodes fail

  Partition Tolerance (P): System continues to operate despite network partitions
    - Network can drop/lose messages between nodes

  Since partitions are inevitable: choose between CP and AP
    - CP: sacrifice availability during partition
    - AP: sacrifice consistency (eventual consistency)

Tradeoffs in Practice:
  CP Systems: Zookeeper, etcd, HBase
  AP Systems: Cassandra, DynamoDB, CouchDB
  CA (rare, impossible in practice): single-node databases
```

3.2 PACELC Theorem

```
PACELC extends CAP:
  If Partition (P):
    - Choose Availability (A) or Consistency (C)

  Else (E) - no partition:
    - Choose Latency (L) or Consistency (C)

Systems classified by PACELC:
```

```
  - DynamoDB: AP/EL (eventual consistency, low latency when no partition)
  - Cassandra: AP/EL
  - BigTable: CP/EC (strong consistency, high latency during normal ops)
  - MongoDB: CP/EL (configurable)
  - MySQL: CP/EC (traditional RDBMS)
```

3.3 Consistency Models

```
Strong Consistency:
  - After write completes, all subsequent reads see that value
  - Implementation: synchronous replication, quorum reads/writes
  - Cost: higher latency, lower availability during partitions
  - Systems: Single-node DB, Zookeeper, etcd, Spanner (TrueTime)

Eventual Consistency:
  - Given enough time without updates, all replicas converge
  - Temporary: stale reads possible
  - Implementation: async replication, gossip protocol
  - Systems: DNS, DynamoDB (default), Cassandra

Causal Consistency:
  - Causally related operations seen in order by all nodes
  - Concurrent operations can be seen in different orders
  - Implementation: vector clocks, version vectors
  - Systems: COPS, Orbe

Read-After-Write Consistency:
  - User always sees their own writes
  - Other users might see stale data
  - Implementation: read-your-writes session, sticky sessions
  - Use: social media posts, user profiles

Monotonic Read Consistency:
  - If a user reads value X at time t, later reads return X or newer
  - No "time travel" (reading older data after newer data)
  - Implementation: timestamp-based routing

Sequential Consistency:
  - Operations appear in some order that's consistent across all nodes
  - Each node's operations appear in program order
  - Weaker than strong consistency, stronger than causal

Quorum-Based Consistency:
  N = number of replicas
  W = write quorum (confirm writes from W replicas)
  R = read quorum (read from R replicas)

  W + R > N: strong consistency (quorum overlap)
  W + R ≤ N: eventual consistency

  Typical: N=3, W=2, R=2 (balance of consistency and availability)
  AWS DynamoDB: N=3, W=1 (eventual) or W=3 (strong)
```

3.4 Load Balancing

```
What is Load Balancing?
  Distributes incoming traffic across multiple servers
  - Prevents any single server from being overwhelmed
  - Provides redundancy (failover)
  - Enables horizontal scaling

Types of Load Balancers:
```

```
  Hardware: F5, Citrix NetScaler (enterprise, expensive)
  Software: NGINX, HAProxy, AWS ALB/NLB, Envoy (flexible, cost-effective)
  DNS-based: Round-robin DNS (simple, no health checking)
  Global: Cloudflare, AWS Route 53 (geo-based, multi-region)

Load Balancing Algorithms:
  Round Robin: requests distributed sequentially
    - Simple, works well for uniform workloads

  Least Connections: send to server with fewest active connections
    - Good for variable-length requests

  Weighted Round Robin: servers receive proportional traffic based on weight
    - Accommodates heterogeneous server capacities

  IP Hash: hash of client IP determines server
    - Ensures same client → same server (sticky sessions)
    - Problem: uneven distribution if few clients

  Least Response Time: send to server with lowest response time
    - Complex, requires active monitoring

Layer 4 vs Layer 7 Load Balancing:
  L4 (Transport Layer): based on IP + TCP/UDP ports
    - Faster, less resource intensive
    - Cannot inspect content (no URL-based routing)

  L7 (Application Layer): based on HTTP headers, URL, cookies
    - Slower, more CPU intensive
    - Can do content-based routing, SSL termination
    - Examples: HAProxy, NGINX, AWS ALB

Health Checks:
  Passive: monitor failed connections, mark server as down
  Active: periodically ping health endpoint
  - /health, /ready endpoints
  - Circuit breaker: after N failures, stop sending traffic

Sticky Sessions (Session Affinity):
  - Same client always goes to same server
  - Implementation: cookie-based (LB sets cookie), IP hash
  - Problem: uneven load, reduced fault tolerance
  - Better approach: externalize session state (Redis, Memcached)
```

3.5 Caching

```
Why Cache?
  - Reduce latency: in-memory reads are microseconds vs database milliseconds
  - Reduce database load: fewer queries = fewer resources
  - Handle traffic spikes: absorb sudden increases in read traffic

Caching Strategies:

  Cache-Aside (Lazy Loading):
    Read: check cache → miss → read DB → update cache → return
    Write: update database → invalidate cache

    Pros: cache only hot data, resilient to cache failure
    Cons: cache miss penalty (3 round trips), stale data possible
    Use: general purpose, most common pattern
```

```
  Read-Through:
    Cache sits between app and database
    Read: cache reads from DB on miss (cache does the loading)
    Pros: simplified app logic, consistent loading
    Cons: cache must know how to load data

  Write-Through:
    Write: write to cache first → cache writes to DB synchronously
    Pros: cache always consistent with DB
    Cons: write latency increased, lower write throughput
    Use: write-heavy systems needing consistency

  Write-Behind (Write-Back):
    Write: write to cache → acknowledge immediately → async write to DB
    Pros: very fast writes, absorbs write bursts
    Cons: data loss if cache fails before DB write
    Use: high-write throughput, non-critical data

  Refresh-Ahead:
    Cache proactively refreshes entries before expiry
    Predict: refresh when entry is close to expiry and frequently accessed
    Pros: reduces latency for hot keys
    Cons: cache may refresh rarely-accessed data

Cache Eviction Policies:
  LRU (Least Recently Used): evict items used longest ago
    - Implemented with doubly-linked list + hash map (O(1))
    - Good: temporal locality

  LFU (Least Frequently Used): evict least frequently accessed
    - Problem: older items with once-high frequency stay forever
    - Solution: logarithmic LFU, windowed LFU

  FIFO (First In First Out): evict oldest items
    - Simple but doesn't consider access patterns

  TTL (Time To Live): expire after fixed duration
    - Simple, predictable
    - Problem: cache stampede (many misses at TTL expiry)

Cache Invalidation:
  - TTL-based: simplest, may serve stale data
  - Write-through: strong consistency
  - Publish/subscribe: notify cache to invalidate on DB update
  - Version-based: increment version on update, compare on read

Distributed Caching:
  Redis:
    - In-memory data structure store
    - Data types: strings, hashes, lists, sets, sorted sets, bitmaps, hyperloglogs
    - Persistence: RDB (snapshot), AOF (append-only file)
    - Replication: master-slave, sentinel for HA
    - Clustering: automatic sharding, 16384 hash slots
    - Pub/Sub, Lua scripting, transactions
    - Use: caching, session store, rate limiting, real-time leaderboards

  Memcached:
    - Simple in-memory key-value store
    - Multi-threaded (vs Redis single-threaded)
    - No persistence, no data structures beyond strings
```

```
    - Uses slab allocation (memory-efficient)
    - Use: simple caching, session storage

  CDN (Content Delivery Network):
    - Caches static content at edge locations
    - Popular: CloudFlare, Akamai, AWS CloudFront, Fastly
    - Types: Push CDN (upload content), Pull CDN (fetch on first request)
    - Benefits: reduced latency, offload origin servers
    - Cache static: images, CSS, JS, video
    - Dynamic content: edge computing (CloudFlare Workers, Lambda@Edge)
```

3.6 Proxies

```
Forward Proxy (Client-side):
  Client requests → Forward Proxy → Internet → Server
  - Hides client IP (anonymity)
  - Bypasses restrictions (geo-blocking)
  - Content filtering
  - Caching (corporate proxy)

Reverse Proxy (Server-side):
  Client → Internet → Reverse Proxy → Server
  - Hides server details (security)
  - Load balancing
  - SSL termination (decrypt HTTPS requests)
  - Caching static content
  - Compression (gzip/brotli)
  - Rate limiting
  - Request filtering (WAF)

  Popular: NGINX, HAProxy, Apache Traffic Server, Envoy, Traefik
```

3.7 Rate Limiting

```
Why Rate Limit?
  - Prevent DoS attacks
  - Ensure fair usage across tenants
  - Protect backend systems from overload
  - Control costs (API usage tiers)

Algorithms:

  Token Bucket:
    - Tokens added at constant rate (r per second)
    - Bucket capacity = burst size (b)
    - Each request consumes 1 token
    - If no tokens, request is dropped/queued
    - Pros: allows bursts, simple
    - Cons: memory per user

  Leaky Bucket:
    - Queue of requests processed at constant rate
    - If queue full, request is dropped
    - Pros: smooths out bursts, predictable rate
    - Cons: doesn't handle traffic spikes well

  Fixed Window Counter:
    - Count requests in fixed time window (e.g., 100 requests/hour)
    - Reset counter at window boundary
    - Pros: simple, memory efficient
    - Cons: traffic spikes at window boundaries
```

```
  Sliding Window Log:
    - Maintain timestamp log of requests
    - Count requests in sliding time window
    - Pros: accurate, no spikes
    - Cons: memory heavy (stores all timestamps)

  Sliding Window Counter:
    - Hybrid: fixed window + overlapping window
    - Approximate rate based on previous window's overlap
    - Pros: memory efficient, smooth
    - Cons: approximate, not exact

Implementation:
  - Client-side: less accurate, can be bypassed
  - Server-side: accurate, more overhead
  - Distributed: Redis + Lua scripting for atomic operations
  - Rate limit headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
  - Response: 429 Too Many Requests + Retry-After header
```

---

4. PHASE 2: STORAGE & DATABASES

Duration: 3-4 weeks

4.1 SQL vs NoSQL

```
SQL (Relational Databases):
  Characteristics:
    - Structured schema (tables, rows, columns)
    - ACID transactions
    - Strong consistency
    - Complex queries via JOINs
    - Vertical scaling (scale up)

  Examples: MySQL, PostgreSQL, Oracle, SQL Server, SQLite

  Best for:
    - Structured data with relationships
    - Transactions requiring ACID (banking, e-commerce)
    - Complex queries and reporting
    - When data integrity is critical

NoSQL Databases:

  Document Stores (MongoDB, CouchDB):
    - Schemaless: store JSON/BSON documents
    - Flexible schema, easy to evolve
    - Rich queries (but not as powerful as SQL JOINs)
    - Best for: content management, catalogs, user profiles

  Key-Value Stores (Redis, DynamoDB):
    - Simple key → value lookup
    - Extremely fast, O(1) access
    - Best for: caching, session store, real-time data

  Column-Family Stores (Cassandra, HBase):
    - Data organized by column families
    - Wide-column: sparse data, flexible schema
    - Highly scalable, write-optimized
    - Best for: time-series data, IoT, analytics
```

```
  Graph Databases (Neo4j, Amazon Neptune):
    - Nodes, edges, properties
    - Efficient for relationship-heavy queries
    - Best for: social networks, recommendation, fraud detection

  Search Engines (Elasticsearch):
    - Full-text search, inverted index
    - Log analysis, monitoring
    - Best for: text search, analytics

SQL vs NoSQL Decision:

  Choose SQL when:
    - Data is highly structured and relational
    - You need ACID transactions
    - Complex queries with JOINs and aggregations
    - Write concurrency is moderate

  Choose NoSQL when:
    - Schema is flexible or evolving
    - Need horizontal scaling (sharding)
    - High write throughput required
    - Simple key-based lookups
    - Handling large volumes of unstructured data

  Polyglot Persistence: Use multiple databases for different needs
    - PostgreSQL: core transactional data
    - Redis: caching, real-time features
    - Elasticsearch: search functionality
    - Cassandra: event logging, time-series
```

4.2 Database Indexing

```
What is an Index?
  - Data structure that speeds up data retrieval
  - Tradeoff: slower writes (index maintenance), more storage

Types of Indexes:

  B-Tree Index (default in most databases):
    - Balanced tree structure: O(log n) search, insert, delete
    - Pages: tree node containing keys + pointers
    - Range queries efficient: in-order traversal of leaves
    - Covering index: includes all columns needed by query

  Hash Index:
    - O(1) lookup for equality searches
    - No range query support
    - Collision handling: chaining, open addressing
    - Used for: in-memory tables, key-value lookups

  Bitmap Index:
    - Array of bits for each distinct value
    - Space-efficient for low-cardinality columns
    - Fast for AND/OR operations
    - Used in: data warehousing, analytics

  GiST/GIN Indexes (PostgreSQL):
    - GiST: generalized search tree (geometry, full-text)
    - GIN: generalized inverted index (arrays, JSONB, full-text)
```

```
  Spatial Index (R-Tree):
    - For geometric/geographic data
    - Bounding boxes, nearest neighbor search
    - Used in: location-based queries, maps

  Inverted Index:
    - Maps terms → documents containing them
    - Used in: full-text search engines (Elasticsearch, Lucene)

Indexing Strategies:
  Clustered Index: data physically ordered by index
    - One per table (or non-existent)
    - Range queries very fast
    - Insert at end for sequential keys

  Non-Clustered Index: separate structure with pointers to data
    - Multiple per table allowed
    - Slightly slower than clustered index
    - Covering index: includes all columns (no table access)

  Composite Index: index on multiple columns
    - Order matters: use query patterns to decide
    - Leftmost prefix rule: index on (A,B,C) covers (A), (A,B), (A,B,C)
    - Does NOT cover (B), (C), (B,C) alone

  Partial Index: index on subset of rows (WHERE clause)
    - Smaller, faster to maintain

  Index-Only Scan: query satisfied entirely from index (no heap access)

When to Index:
  - Columns used in WHERE clauses frequently
  - JOIN columns (foreign keys)
  - ORDER BY columns
  - Columns with high selectivity (many unique values)

When NOT to Index:
  - Small tables (full scan is faster)
  - Columns rarely used in queries
  - Columns with low selectivity (gender, boolean)
  - Columns frequently updated (write overhead)
  - Tables with heavy write load
```

4.3 Database Replication

```
Why Replicate?
  - High availability (survive node failures)
  - Read scalability (distribute read load)
  - Geographic locality (serve data closer to users)
  - Analytics without impacting production

Single Leader (Master-Slave / Primary-Replica):
  - One node accepts writes (leader)
  - Leader sends changes to followers (replicas)
  - Followers serve reads (read scalability)

  Synchronous Replication:
    - Leader waits for follower(s) to confirm write
    - Pros: no data loss on failure
    - Cons: slower writes (highest latency among followers)
```

```
  Asynchronous Replication:
    - Leader doesn't wait for follower confirmation
    - Pros: fast writes
    - Cons: data loss on leader failure (last writes not replicated)

  Semi-Synchronous:
    - Leader waits for at least 1 follower to confirm
    - Balance: some durability, some speed

  Failover:
    - Leader fails → promote a follower to leader
    - Automatic: monitoring system detects failure, promotes
    - Manual: DBA intervenes
    - Split brain: both old and new leader accept writes (dangerous!)
    - Solutions: fencing, lease-based, consensus

Multi-Leader Replication:
  - Multiple nodes accept writes
  - Each leader replicates to others
  - Conflict resolution: last-write-wins, CRDTs, custom merge
  - Use: multi-datacenter, offline-first apps (CouchDB)

Leaderless Replication (Dynamo-style):
  - Any node accepts writes (DynamoDB, Cassandra)
  - Read repair: detect stale data on read and fix
  - Anti-entropy: background process fixes inconsistencies
  - Quorum: N, W, R parameters

Replication Lag Problems:
  - Read-after-write: user doesn't see their own write
    Fix: read from leader for user's data, route based on timestamp
  - Monotonic read: user sees data go backwards
    Fix: route same user to same replica
  - Consistent prefix: see writes in wrong order
    Fix: use causal consistency, version vectors
```

4.4 Database Sharding

```
What is Sharding?
  - Horizontal partitioning: split data across multiple databases
  - Each shard contains a subset of the data
  - Shards are independent (no shared resources)

Sharding Strategies:

  Range-Based Sharding:
    - Divide data by key range (e.g., user_id 1-10000 → shard 1, 10001-20000 →
    shard 2)
    - Pros: simple, range queries efficient
    - Cons: hot spots (new users go to last shard), uneven distribution

  Hash-Based Sharding:
    - Hash key → shard_id = hash(key) % num_shards
    - Pros: even distribution
    - Cons: resharding expensive (rehashing all keys), range queries across shards
    - Solution: consistent hashing (minimal reshuffling on resize)

  Directory-Based Sharding:
    - Maintain lookup table mapping key → shard
    - Pros: flexible, can move shards easily
    - Cons: single point of failure, lookup overhead
```

```
  Geographic Sharding:
    - Shard by region (users in US → US shard, EU → EU shard)
    - Pros: low latency for local users, regulatory compliance (GDPR)
    - Cons: uneven distribution, user mobility

Sharding Challenges:
  Resharding: adding/removing shards is expensive
    - Pre-split: create many shards, then merge
    - Consistent hashing: minimize data movement
    - Virtual nodes: each physical node handles multiple virtual nodes

  Cross-Shard Queries:
    - JOINs across shards: inefficient (gather data from all shards)
    - Solution: denormalize, fan-out queries, map-reduce pattern

  Distributed Transactions:
    - 2PC (Two-Phase Commit): coordinator + participants
    - Phase 1: prepare → all agree
    - Phase 2: commit → all commit or rollback
    - Limitations: coordinator is SPOF, blocking protocol
    - Alternatives: Saga pattern, TCC (Try-Confirm/Cancel)

  Unique Identifiers Across Shards:
    - UUID: globally unique, not ordered
    - Snowflake ID: timestamp + worker ID + sequence
      - 64-bit: 1 sign bit + 41 timestamp (69 years) + 10 worker (1024 nodes) +
      12 sequence (4096/s)
    - Database sequence: central authority (SPOF)
    - Ticket server: simple DB-based ID generation

  Consistent Hashing:
    - Hash nodes and keys onto a ring (0 to 2^n - 1)
    - Key assigned to next node clockwise on ring
    - Adding/removing node: only adjacent keys reshuffled
    - Virtual nodes: each physical node appears as multiple virtual nodes on
    ring (better distribution)
    - Used in: DynamoDB, Cassandra, Discord, Akamai CDN
```

4.5 Block Storage, File Storage, Object Storage

```
Block Storage (EBS, SSD, HDD):
  - Raw storage volumes presented to OS
  - Accessed as block device (mount point)
  - Application: databases, VMs
  - Low latency, high performance
  - Max size: ~16TB per volume
  - Backup: snapshots

File Storage (EFS, NFS, SMB):
  - Hierarchical file system with directories
  - Accessed via file paths, standard file operations
  - Shared across multiple instances simultaneously
  - Application: shared files, content management
  - Scaling: up to petabytes, thousands of concurrent connections

Object Storage (S3, GCS, Azure Blob):
  - Flat namespace (buckets + objects)
  - Each object: data + metadata + unique ID
  - Accessed via REST API
  - Virtually unlimited storage
```

```
  - Application: static files, backups, data lakes
  - Durability: 99.999999999% (11 nines)
  - Cost: low, tiers (standard, infrequent access, archive)

Comparison:
  | Feature | Block | File | Object |
  |---|---|---|---|
  | Performance | Fastest | Moderate | Moderate |
  | Scalability | Limited | Moderate | Unlimited |
  | Cost | High | Moderate | Low |
  | Access | OS-level | File path | HTTP API |
  | Use | Databases, VMs | Shared files | Backups, web content |
  | Consistency | Strong | Strong | Eventual (S3 strong) |
```

---

5. PHASE 3: COMMUNICATION & APIS

Duration: 2-3 weeks

5.1 REST API Design

```
Principles:
  - Stateless: each request contains all needed info
  - Resource-based: nouns, not verbs (/users not /getUsers)
  - Uniform interface: standard HTTP methods + status codes
  - HATEOAS: responses include links to related resources

HTTP Methods:
  GET: retrieve resource (idempotent, safe)
  POST: create resource (non-idempotent)
  PUT: replace resource (idempotent)
  PATCH: partial update (can be idempotent)
  DELETE: remove resource (idempotent)

URL Structure:
  POST   /users                    # Create user
  GET    /users                    # List users
  GET    /users/:id                # Get user by ID
  PUT    /users/:id                # Replace user
  PATCH  /users/:id                # Update user partially
  DELETE /users/:id                # Delete user
  GET    /users/:id/orders         # User's orders
  GET    /users/:id/orders/:oid    # Specific order

Query Parameters:
  GET /users?page=1&limit=20&sort=created_at:desc
  GET /users?filter=status:active,role:admin
  GET /users?fields=id,name,email

Pagination:
  Offset-based: GET /items?offset=0&limit=20
    - Problems: inconsistent if data changes, page drift

  Cursor-based: GET /items?cursor=abc123&limit=20
    - Cursor = encoded position (usually ID or timestamp)
    - Consistent even with inserts/deletes
    - Cannot jump to arbitrary page

Versioning:
  URL: /api/v1/users
  Header: Accept: application/vnd.myapp.v1+json
```

```
  Parameter: /api/users?version=1

Error Handling:
  {
    "error": {
      "code": "USER_NOT_FOUND",
      "message": "User with ID 123 not found",
      "details": { "userId": "123" }
    }
  }

  Status codes:
    200 OK, 201 Created, 204 No Content
    400 Bad Request, 401 Unauthorized, 403 Forbidden
    404 Not Found, 409 Conflict, 422 Unprocessable Entity
    429 Too Many Requests
    500 Internal, 502 Bad Gateway, 503 Service Unavailable

Idempotency:
  - POST /payments with Idempotency-Key header
  - Same key → same result (dedup)
  - Critical for financial transactions

Security:
  - HTTPS everywhere
  - Authentication: JWT, OAuth 2.0, API keys
  - Rate limiting (per user, per IP)
  - Input validation (never trust client)
  - CORS configuration
  - SQL injection prevention (parameterized queries)
  - Request size limits
```

5.2 gRPC

```
What is gRPC?
  - High-performance RPC framework by Google
  - Uses Protocol Buffers for serialization
  - Built on HTTP/2 (multiplexed, bidirectional streaming)
  - Language-agnostic: generate client/server code from .proto

Protocol Buffers:
  syntax = "proto3";

  service UserService {
      rpc GetUser (GetUserRequest) returns (User);
      rpc ListUsers (ListUsersRequest) returns (stream User);
      rpc UpdateUser (stream UpdateUserRequest) returns (User);
      rpc Chat (stream ChatMessage) returns (stream ChatMessage);
  }

  message User {
      string id = 1;
      string name = 2;
      string email = 3;
      int64 created_at = 4;
  }

  message GetUserRequest {
      string id = 1;
  }
```

```
Types of gRPC Calls:
  - Unary: client sends 1 request, gets 1 response
  - Server Streaming: client sends 1, gets stream of responses
  - Client Streaming: client sends stream, gets 1 response
  - Bidirectional Streaming: both sides stream simultaneously

REST vs gRPC:
  | Feature | REST | gRPC |
  |---|---|---|
  | Protocol | HTTP/1.1 or HTTP/2 | HTTP/2 |
  | Serialization | JSON/XML (text) | Protobuf (binary, smaller) |
  | Performance | Slower | 7-10x faster |
  | Streaming | SSE, WebSocket | Native |
  | Browser support | Native | Needs gRPC-Web proxy |
  | Readability | Human-readable | Binary (use grpcurl) |
  | Code generation | Manual/Swagger | Built-in from .proto |
  | Caching | HTTP caching | No built-in caching |
  | Tooling | Mature (Postman, curl) | Growing (grpcurl, BloomRPC) |

When to use gRPC:
  - Microservice-to-microservice communication
  - Real-time streaming applications
  - Polyglot environments (multiple languages)
  - Low-latency, high-throughput systems

When to use REST:
  - Public-facing APIs
  - Browser-based applications
  - When caching is important
  - Simple CRUD operations
```

5.3 GraphQL

```
What is GraphQL?
  - Query language for APIs (developed by Facebook, 2015)
  - Clients specify exactly what data they need
  - Single endpoint: /graphql

Schema Definition:
  type User {
      id: ID!
      name: String!
      email: String
      posts: [Post!]!
  }

  type Post {
      id: ID!
      title: String!
      content: String!
      author: User!
      comments: [Comment!]!
  }

  type Query {
      user(id: ID!): User
      users(limit: Int, offset: Int): [User!]!
      search(query: String!): [SearchResult!]!
  }

  type Mutation {
```

```
      createUser(input: CreateUserInput!): User!
      updateUser(id: ID!, input: UpdateUserInput!): User!
      deleteUser(id: ID!): Boolean!
  }

Query Example:
  query GetUserWithPosts {
      user(id: "123") {
          name
          email
          posts {
              title
              content
          }
      }
  }

N+1 Problem in GraphQL:
  - Without batching: each field resolver hits DB
  - Solution: DataLoader (batching + caching)
    - Batch requests: DataLoader collects keys, loads all at once
    - Caches per request: avoids duplicate loads

GraphQL vs REST:
  | Feature | REST | GraphQL |
  |---|---|---|
  | Data fetching | Fixed structure, over/under fetching | Client specifies
  exact fields |
  | Endpoints | Multiple endpoints | Single endpoint |
  | Versioning | URL/Header versioning | Evolve schema, deprecate fields |
  | Caching | HTTP caching (built-in) | Custom caching needed |
  | Complexity | Simple requests | Complex queries impact server |
  | Tooling | Mature | Apollo, Relay, Hasura |

When to use GraphQL:
  - Complex UIs with diverse data requirements
  - Mobile apps (limited bandwidth, need exact data)
  - Rapidly evolving frontends
  - When multiple clients need different data shapes

When NOT to use GraphQL:
  - Simple APIs with fixed data requirements
  - Heavy caching needed
  - Complex file uploads
  - Simple applications (overkill)
```

5.4 Message Queues

```
Why Message Queues?
  - Decoupling: producers don't need to know about consumers
  - Asynchronous communication: non-blocking
  - Load leveling: buffer between traffic spikes and processing
  - Reliability: messages persisted, retries, dead-letter queues
  - Scalability: consumers can be added independently

Message Queue Models:

  Point-to-Point (Queue):
    - Each message consumed by exactly one consumer
    - Load balancing across consumers
    - Messages removed after consumption
```

```
    - Use: task queues, job processing

  Publish-Subscribe (Topic):
    - Each message sent to multiple subscribers
    - Each subscriber gets its own copy
    - Use: event broadcasting, notifications, data replication

Apache Kafka:
  - Distributed streaming platform
  - Topics: categories of messages, partitioned across brokers
  - Partitions: ordered, immutable sequence of messages
  - Offsets: position in partition (consumer tracks offset)
  - Consumer Groups: load-balanced consumption

  Key Concepts:
    - Broker: server in Kafka cluster
    - Topic: message category
    - Partition: ordered log within topic
    - Producer: publishes messages to topics
    - Consumer: subscribes to topics, reads messages
    - Consumer Group: set of consumers sharing load
    - Offset: position index in partition
    - Zookeeper or KRaft: cluster coordination

  Kafka vs RabbitMQ:
    | Feature | Kafka | RabbitMQ |
    |---|---|---|
    | Design | Distributed log | Message broker |
    | Performance | High throughput (msgs/sec) | Moderate |
    | Message model | Pull-based | Push-based |
    | Persistence | Durable (disk) | Configurable |
    | Ordering | Per-partition | Per-queue |
    | Routing | Topic-based | Topic, direct, headers, fanout |
    | Latency | Higher (milliseconds) | Lower (microseconds) |
    | Message retention | Configurable time | Removed after ack |
    | Use cases | Event sourcing, streaming | Task queues, RPC |

  When to use Kafka:
    - High-throughput event streaming
    - Log aggregation and monitoring
    - Event sourcing / CQRS
    - Stream processing (Kafka Streams, ksqlDB)
    - Data integration (CDC pipeline)

  When to use RabbitMQ:
    - Reliable task queue
    - Complex routing scenarios
    - Low-latency messaging
    - Need message acknowledgments and retries

Cloud Managed Queues:
  AWS SQS: Simple Queue Service
    - Fully managed, unlimited throughput
    - Standard: at-least-once delivery, high throughput
    - FIFO: exactly-once, ordered, 3000 msg/s with batching

  AWS SNS: Simple Notification Service
    - Pub/sub messaging
    - Fan-out to SQS, Lambda, email, SMS, HTTP
```

```
Google Pub/Sub:
  - Global, at-least-once
  - Pull and push subscriptions
  - Exactly-once with subscriber ack
```

---

6. PHASE 4: DISTRIBUTED SYSTEMS

Duration: 3-4 weeks

6.1 Microservices Architecture

```
Monolith vs Microservices:
  Monolith:
    - Single deployable unit
    - Pros: simple development, easy testing, low latency
    - Cons: scales as whole, deployment risk, tech lock-in

  Microservices:
    - Independent services with bounded contexts
    - Pros: independent scaling, deployment, technology diversity
    - Cons: distributed complexity, network latency, data consistency

When to Use Microservices:
  - Large team (2-pizza teams, each owns services)
  - Complex domain with clear boundaries
  - Need independent scaling of components
  - Different technologies for different parts

Patterns:

  Service Discovery:
    - Clients need to find service instances dynamically
    - Client-side: client queries registry (Netflix Eureka)
    - Server-side: load balancer queries registry (Kubernetes DNS)
    - Tools: Consul, etcd, Zookeeper, Eureka, Kubernetes DNS

  API Gateway:
    - Single entry point for all clients
    - Responsibilities: routing, auth, rate limiting, caching, logging
    - Can aggregate responses from multiple services
    - Tools: Kong, NGINX + Lua, AWS API Gateway, Zuul, Envoy

  Circuit Breaker:
    - Failed service → stop calling it → fail fast
    - States: Closed (normal), Open (failing), Half-Open (testing)
    - When Open: requests fail immediately (timeout saved)
    - After timeout: Half-Open, allow test request
    - If test passes: Close, else: stay Open
    - Tools: Hystrix (Netflix, deprecated), Resilience4j, Sentinel

  Saga Pattern:
    - Distributed transaction across services
    - Each service executes local transaction + publishes event
    - Next service triggered by event
    - If failure: compensating transactions roll back

    Choreography: services communicate via events
      - Decoupled, no coordinator
      - Complex to trace
```

```
    Orchestration: central coordinator (Saga Manager)
      - Easier to monitor and manage
      - Coordinator is SPOF

  CQRS (Command Query Responsibility Segregation):
    - Separate commands (writes) from queries (reads)
    - Different models for reading and writing
    - Can have separate databases (read replica, optimized for queries)
    - Eventual consistency between command and query sides
    - Use: complex domains, high read vs write imbalance

  Event Sourcing:
    - Store state changes as event log (not current state)
    - Current state = fold over all events
    - Benefits: audit trail, temporal queries, rebuild state
    - Challenges: event schema evolution, storage, querying current state

Microservice Communication:
  Synchronous: REST, gRPC
    - Request/response, immediate
    - Coupling: caller waits, cascading failures

  Asynchronous: Message queues, events
    - Decoupled, eventual consistency
    - Complex error handling
```

6.2 Consensus Algorithms

```
What is Consensus?
  - Distributed agreement on a value/state
  - Properties: agreement (all decide same), validity (decided value proposed),  ↩
  termination (eventually decide)

Two-Phase Commit (2PC):
  Coordinator → all participants: "prepare"
  Participants → coordinator: "ready" or "abort"
  Coordinator → all: "commit" or "abort"

  Problems:
    - Blocking: coordinator failure blocks everyone
    - No fault tolerance: coordinator SPOF
    - Not suitable for high-availability systems

Paxos:
  Roles: Proposer, Acceptor, Learner, Leader
  Phase 1 (Prepare):
    Proposer → Acceptors: prepare(n)
    Acceptors → Proposer: promise(n, last_accepted_n, last_accepted_v)

  Phase 2 (Accept):
    If majority promises → Proposer → Acceptors: accept(n, v)
    If majority accepts → consensus reached

  Problems: difficult to understand, implement, slow under normal operation

Raft (Designed for Understandability):
  Leader Election:
    - Servers start in follower state
    - If no leader heartbeat in election timeout → become candidate
    - Request votes from other servers
    - Majority votes → become leader
```

```
    - Terms: monotonic increasing, each term has at most 1 leader

  Log Replication:
    - Leader receives client requests
    - Appends to log, sends AppendEntries to followers
    - Majority confirm → commit → apply to state machine
    - Follower crashes: leader retries indefinitely

  Safety:
    - Election restriction: candidate must have latest log entries
    - Commitment: only commit entries from current term
    - Leader completeness: committed entries are in all future leaders' logs

  Used in: etcd, Consul, TiKV, MongoDB (Raft-based)

ZAB (Zookeeper Atomic Broadcast):
  - Used in Apache Zookeeper
  - Phases: discovery, synchronization, broadcast
  - Similar to Raft but designed for Zookeeper's use case

Gossip Protocol:
  - Decentralized, epidemic-style communication
  - Each node periodically exchanges info with random peer
  - Info spreads exponentially: O(log n) rounds to reach all nodes
  - Properties: scalable, fault-tolerant, eventual consistency
  - Used in: Cassandra, DynamoDB, Consul, Redis Cluster
  - Failure detection: SWIM protocol (Scalable Weakly-consistent Infection-style)

Quorum:
  - Minimum number of nodes that must agree
  - Read quorum (R) + Write quorum (W) > N (replicas)
  - Without partition: any majority works
  - With partition: strict majority (N/2 + 1)
  - Grid quorum: more complex but better fault tolerance
```

6.3 Distributed System Patterns

```
Leader Election:
  - Pick one node as leader (for coordination)
  - Implementation: smallest ID, Raft, Paxos, Zookeeper ephemeral znodes
  - All nodes know who is leader
  - If leader fails, new election triggered

Distributed Locking:
  - Mutual exclusion across machines
  - Implementation: Redis (SETNX), Zookeeper (ephemeral sequential znodes), etcd
  - Redlock algorithm: acquire lock on majority of Redis nodes
  - Problems: clock drift, garbage collection pauses

  Preferred: Single-node Redis + fencing tokens (epoch-based)

Distributed Cache:
  - Redis Cluster: automatic sharding + replication
  - Memcached: simple distributed hash table
  - Consistent hashing for distribution

Distributed Counter:
  - Atomic counter across machines
  - Implementation: Redis INCR, CRDT counters (PN-Counter)
  - Problem: network delays cause inaccuracies
  - Solution: approximate counters (HyperLogLog)
```

```
Bloom Filter:
  - Probabilistic data structure for membership testing
  - Returns: "definitely not in set" or "probably in set"
  - False positive possible, false negative impossible
  - Operations: add, check (no delete)
  - Parameters: n (expected elements), p (false positive rate) → m (bits), k (hash functions)
  - Use: caching (avoid cache miss storms), spell check, blockchain (SPV)

  Counting Bloom Filter: supports delete (counters instead of bits)

Probabilistic Data Structures:
  HyperLogLog: count distinct elements (~1% error) using very little memory
    - 12KB → count billions of unique elements

  Count-Min Sketch: frequency estimation
    - Underestimates (overcounts due to collisions)

  Skip List: probabilistic balanced tree
    - O(log n) average, simpler than balanced trees
    - Used in: Redis sorted sets, LevelDB memtable
```

6.4 Monitoring & Observability

```
Three Pillars of Observability:

  Logging:
    - Structured logs (JSON format)
    - Levels: DEBUG, INFO, WARN, ERROR, FATAL
    - Correlation IDs: trace requests across services
    - Centralized: ELK Stack, Loki, Splunk
    - Avoid: logging sensitive data (PII, secrets)

  Metrics:
    - Time-series data
    - Types: counters (requests total), gauges (memory), histograms (latency p50/p95/p99)
    - USE Method: Utilization, Saturation, Errors (for resources)
    - RED Method: Rate, Errors, Duration (for services)
    - Collection: Prometheus, Telegraf, StatsD
    - Visualization: Grafana

  Tracing:
    - End-to-end request tracking across services
    - Spans: unit of work (service call, DB query, cache lookup)
    - Trace = forest of spans (parent-child relationships)
    - Context propagation: trace ID + span ID passed via headers
    - Sampling: head-based (decision at start), tail-based (decision after)
    - Tools: Jaeger, Zipkin, OpenTelemetry, AWS X-Ray

Health Check APIs:
  /health: simple liveness check
  /ready: dependency check (DB, cache, downstream)
  /metrics: prometheus metrics endpoint
  /info: build info, config, dependencies

Incident Response:
  - Detect: alerting (Prometheus AlertManager, PagerDuty)
  - Respond: acknowledge, identify severity
  - Diagnose: dashboards, logs, traces
```

```
  - Resolve: rollback, fix forward
  - Learn: post-mortem (blameless)
```

6.5 Security & Authentication

```
Authentication vs Authorization:
  Authentication: verify identity ("who you are")
  Authorization: verify permissions ("what you can do")

JWT (JSON Web Token):
  Header: {"alg": "RS256", "typ": "JWT"}
  Payload: {"sub": "user123", "role": "admin", "exp": 1699000000}
  Signature: base64(header) + "." + base64(payload) + secret key

  Stateless: server doesn't need to store session
  Problems: cannot revoke (without blacklist), large (HTTP headers)

  Use: short-lived access token, refresh token for renewals

OAuth 2.0:
  Roles: Resource Owner (user), Client (app), Authorization Server, Resource Server
  Flows:
    - Authorization Code: most secure, server-side apps
    - Implicit: deprecated, use PKCE instead
    - Client Credentials: server-to-server
    - Resource Owner Password: legacy, avoid

  PKCE (Proof Key for Code Exchange): public clients (mobile, SPA)
    - Client creates code_verifier (random) + code_challenge (SHA256)
    - Authorization: code_challenge sent
    - Token exchange: code_verifier sent → server verifies

TLS/SSL:
  - Encryption in transit
  - Certificate Authority (CA) verification
  - Handshake: ClientHello → ServerHello + Certificate → Key Exchange → Secure
  Connection
  - Mutual TLS (mTLS): both sides present certificates
  - Used in: service mesh (Istio, Linkerd)

API Security:
  - API keys: identify client (not authentication)
  - Rate limiting: per client, per endpoint
  - Input validation: sanitize all inputs
  - CORS: restrict cross-origin requests
  - CSRF tokens: prevent cross-site request forgery
  - SQL injection: parameterized queries
  - Secrets management: Vault, AWS Secrets Manager, env variables

OWASP Top 10:
  1. Broken Access Control
  2. Cryptographic Failures
  3. Injection (SQL, NoSQL, Command)
  4. Insecure Design
  5. Security Misconfiguration
  6. Vulnerable Components
  7. Auth Failures
  8. Data Integrity Failures
  9. Logging & Monitoring Failures
  10. SSRF (Server-Side Request Forgery)
```

---

7. PHASE 5: HIGH-LEVEL DESIGN CASE STUDIES

7.1 Design a URL Shortener (bit.ly)

```
Requirements:
  - Generate short URL from long URL
  - Redirect short URL to original
  - High availability, low latency redirects
  - Analytics (click count, referrer, timestamp)

API:
  POST /shorten
    Request: {"long_url": "https://example.com/very/long/url"}
    Response: {"short_url": "https://short.ly/abc123", "expires_in": 365}

  GET /abc123
    Redirect 302 → original URL

Capacity Estimation:
  - 100M new URLs/month → ~40/s
  - 10B redirects/month → ~4000/s
  - URL storage: 100M × 500 bytes = 50 GB/month
  - 5 years: 3 TB total

Short URL Generation:
  Base62 encoding: [a-z A-Z 0-9] = 62 characters
  7 chars: 62^7 ≈ 3.5 trillion unique URLs

  Approach 1: Hash + collision detection
    - MD5/SHA-256 → take first 7 chars → check DB
    - Collision unlikely but possible

  Approach 2: Distributed ID → Base62
    - Use Snowflake ID → encode in Base62
    - No collisions, ordered, scalable

  Approach 3: Pre-generated keys
    - Key Generation Service (KGS) pre-generates keys
    - Store unused keys in DB
    - Assign on demand (fast allocation)

Architecture:
  Client → Load Balancer → Web Servers
    → Cache (Redis): hot short URLs → redirect without DB hit
    → DB (Cassandra/DynamoDB): key-value storage (short → long)
    → Analytics Pipeline: Kafka → Spark → DB

  Redirect path:
    1. Client requests GET /abc123
    2. Web server checks Redis cache
    3. If miss, check DB
    4. Cache result in Redis (TTL)
    5. Return 302 redirect

  Write path:
    1. Client POST long URL
    2. Server checks cache (avoid duplicates, optional)
    3. Generate short key
    4. Store in DB: (short_key, long_url, created_at, user_id)
```

```
    5. Store in cache
    6. Return short URL

Scaling:
  - Web tier: stateless, scale horizontally
  - Cache: Redis cluster with replicas
  - DB: range-based sharding on short_key
  - Analytics: batch process logs, update counters periodically
```

7.2 Design WhatsApp / Messenger

```
Requirements:
  - One-on-one and group messaging
  - Delivery receipts (sent, delivered, read)
  - Online/offline status
  - Media sharing (images, video, documents)
  - End-to-end encryption
  - 1B+ users, billions of messages/day

API:
  - WebSocket: persistent connection for real-time messaging
  - REST: history, profile, settings

Messages:
  Message format:
    {
      "message_id": "uuid",
      "sender_id": "user123",
      "conversation_id": "conv456",
      "content": "Hello!",
      "message_type": "text|image|video|document",
      "timestamp": 1699000000
    }

Architecture:
  Client → WebSocket LB → Chat Service
    → Message Queue (Kafka): buffer messages for async processing
    → Message Store (Cassandra): time-series, write-optimized
    → User Presence Service (Redis): online status, last seen
    → Notification Service (APNs/FCM): push notifications for offline users

  Delivery Flow:
    1. User A sends message to User B
    2. Chat service receives via WebSocket
    3. Store message in DB (acknowledge saved)
    4. Check if User B online (presence service)
    5. If online: push via WebSocket
    6. If offline: store for later delivery + push notification
    7. User B receives → send delivery receipt
    8. User B reads → send read receipt

Storage Schema:
  Messages table (Cassandra):
    conversation_id (partition key)
    message_id (clustering key, timeUUID)
    sender_id, content, type, timestamp, metadata

  Conversations table (MySQL/PostgreSQL):
    conversation_id, type (1-on-1/group), created_at, last_message, last_activity

  Participants table:
```

```
    conversation_id, user_id, last_read_message_id

Scaling:
  - WebSocket servers: sticky connections (L4 LB)
  - Chat service: shard by conversation_id (consistent hashing)
  - Message DB: time-based partitioning (Cassandra compaction)
  - Presence: Redis Cluster, user → server mapping
  - Group messaging: fan-out on write (small groups) vs fan-out on read (large groups)
  - Media: CDN for images/videos, separate upload service
  - Encryption: Signal Protocol (end-to-end, X3DH + Double Ratchet)
```

7.3 Design Twitter

```
Requirements:
  - Users post tweets (280 chars)
  - Users follow other users
  - Timeline: home timeline (tweets from followed) + user timeline (own tweets)
  - Like, retweet, reply
  - Trending topics
  - 500M users, 500M tweets/day

Capacity Estimation:
  - Tweets/day: 500M → ~6000/s peak
  - Reads/day: 20B timeline views → ~230K/s
  - Storage: 500M × 1KB = 500 GB/day
  - Photos/videos: separate storage (CDN)

Architecture:

  Tweet Writing:
    User → Load Balancer → Write API → Tweet Service
      → Tweet DB (Cassandra): tweet_id, user_id, content, timestamp, metadata
      → Fan-out Service: distribute to followers' timelines

  Fan-out Strategies:

    Fan-out on Write (Push):
      - When user tweets, push to all followers' timelines
      - Timeline = list of tweet IDs in Redis
      - Pros: fast reads (just fetch sorted list)
      - Cons: celebrity users (millions of followers → massive fan-out)

    Fan-out on Read (Pull):
      - When user views timeline, fetch tweets from followed users
      - Merge and sort on the fly
      - Pros: no fan-out for celebrities, lighter writes
      - Cons: slow reads (query many users)

    Hybrid:
      - Celebrities (>100K followers): fan-out on read
      - Regular users: fan-out on write
      - Threshold configurable

  Timeline Generation:
    Home Timeline (Redis sorted set):
      - Key: timeline:{user_id}
      - Members: tweet_ids
      - Score: timestamp (for sorting)
      - TTL: 7 days (older tweets from DB)
      - Pagination: ZREVRANGE key start stop
```

```
    User Timeline:
      - Simple: SELECT * FROM tweets WHERE user_id = {uid} ORDER BY timestamp DESC
      - Cache in Redis for speed

  Trending Topics:
    - Stream processing (Kafka Streams / Flink)
    - Count hashtags in sliding window (1 hour, 24 hours)
    - Normalize: remove duplicates, filter spam
    - Store in Redis sorted sets (topic → count)
    - Update periodically (every 5 minutes)

  Search:
    - Elasticsearch cluster
    - Inverted index on tweet text
    - Index tweets in near real-time (Kafka → Elastic)
    - Query: text, user, date range, hashtags

Caching Strategy:
  - User profiles: Redis, TTL 1 hour
  - Timeline: Redis sorted set, TTL 7 days
  - Tweet content: Redis, TTL 24 hours
  - Trending topics: Redis sorted set, updated every 5 min
  - CDN: images, videos

Scaling:
  - Tweet IDs: Snowflake (timestamp + shard ID + sequence)
  - Sharding: by user_id (user timeline) and tweet_id (tweet storage)
  - Read replicas: for DB queries
  - Cache: Redis Cluster, shard by user_id
```

7.4 Design YouTube / Video Streaming

```
Requirements:
  - Upload videos (any format, any size)
  - Video streaming with adaptive bitrate
  - View counts, likes, comments, subscriptions
  - Search videos
  - 2B+ monthly active users

Capacity Estimation:
  - Video uploads: 500 hours/minute
  - Storage: ~50 MB/minute for HD → 500 hours = 1.5 TB/minute
  - Daily watch: 1B hours → ~1M concurrent viewers at peak

Architecture Overview:

  Upload Flow:
    User → Load Balancer → Upload Service
      → Video Metadata DB (MySQL, sharded)
      → Original Video Storage (S3/blob)
      → Async Processing Pipeline

  Processing Pipeline:
    S3 Event Notification → Message Queue (Kafka)
    → Transcoder Workers (distributed encoding farm)
    → Thumbnail Generator
    → Content Moderation (AI-based)
    → CDN Upload

  Streaming Flow:
```

```
    User → CDN (edge server)
    → If miss, origin server fetches from S3
    → Adaptive bitrate streaming (DASH/HLS)

Video Transcoding:
  Why: different devices need different formats/resolutions
  Resolutions: 144p, 240p, 360p, 480p, 720p, 1080p, 4K, 8K
  Codecs: H.264 (widest support), H.265/HEVC (better compression), VP9 (Google), AV1 (open)

  Segmented Streaming:
    - Video split into chunks (2-10 seconds)
    - Manifest file: lists available streams + segments
    - Player switches quality based on bandwidth
    - HLS (Apple): .m3u8 manifest + .ts segments
    - MPEG-DASH: .mpd manifest + .m4s segments

  Processing a 1080p video:
    1. Split into segments (2s chunks)
    2. Generate thumbnail at multiple points
    3. Transcode to multiple resolutions
    4. Package into DASH/HLS format
    5. Upload to CDN
    6. Update metadata (duration, sizes, thumbnails)

CDN Strategy:
  - Edge servers cache video segments
  - Cache policy: long TTL (videos rarely change)
  - Geo-distributed: serve from nearest edge
  - Pre-fetch: predict and cache popular content
  - Tiered caching: edge → regional → origin

Scaling:
  - Upload service: message queue + workers (auto-scale based on queue depth)
  - Transcoder: auto-scaling group of spot instances
  - Metadata DB: shard by video_id
  - View counts: eventual consistency (Redis pipelines → batch write to DB)
  - Comments: Cassandra (time-series, per video)
  - Search: Elasticsearch
  - Recommendations: ML pipeline (offline training, real-time serving)
```

7.5 Design Uber / Ride-Hailing

```
Requirements:
  - Riders request rides
  - Drivers accept/decline rides
  - Real-time driver/rider location tracking
  - Matching algorithm: closest available driver
  - Pricing: dynamic/surge pricing
  - Trip history, payments, ratings

Architecture:

  Location Ingestion:
    Driver's phone → WebSocket → Location Service
      → Kafka (locomotive events stream)
      → Stream processor (update location in Redis grid)
      → Analytics pipeline (trip analytics, heatmaps)

  Ride Request Flow:
    1. Rider requests ride (pickup, destination)
```

```
    2. Ride Service creates ride request
    3. Search Service finds nearby drivers (Redis geospatial)
    4. Dispatch Service sends request to top N drivers
    5. First driver to accept gets the ride
    6. Notify rider and driver

Geospatial Indexing:
  - Redis Geospatial: GEOADD, GEORADIUS (O(log N))
    - Flat-earth approximation, good for short distances
  - Grid-based: divide world into cells, maintain driver count per cell
    - Pre-compute head counts for surge pricing
  - Uber's H3: hexagonal hierarchical geospatial indexing
    - Hexagons: more uniform than squares
    - Resolution: 16 levels (meters to global)
    - Cell ID: 64-bit integer (fast indexing)

  Finding nearby drivers:
    Redis: GEOADD drivers:loc {lat} {lng} {driver_id}
    Search: GEORADIUS drivers:loc {lat} {lng} 5 km

Dispatch Algorithm:
  1. Find closest 10-20 drivers
  2. Estimate ETAs for each (from routing service)
  3. Filter: deadheading (distance to dispatch)
  4. Filter: driver state (not on trip, not paused)
  5. Send request to top N drivers
  6. Each driver has ~15 seconds to accept
  7. If all decline: expand radius, find more drivers

Surge Pricing:
  - Real-time supply/demand imbalance
  - Input: # riders looking (demand) vs # available drivers (supply) per hex cell
  - Multiplier: higher where demand > supply
  - Notify riders before booking
  - Encourages drivers to move to high-demand areas

Trip Lifecycle:
  1. REQUESTED: rider requests, searching drivers
  2. ACCEPTED: driver accepts
  3. ARRIVED: driver at pickup
  4. IN_PROGRESS: en route to destination
  5. COMPLETED: trip ended
  6. CANCELLED: either party cancels

  Each state change: event → Kafka → downstream services
    - Payment Service: compute fare
    - Rating Service: prompt for ratings
    - Analytics: compute trip metrics
    - Invoice: generate receipt

Scaling:
  - Location writes: high volume (every 3-4 seconds per driver)
    - Batch in-memory, flush to Redis per second
    - Compress: use delta encoding for GPS coordinates
  - Redis: cluster mode, shard by geo-region
  - Kafka: partition by region (EU, US, APAC)
  - Matching: in-memory computation (low latency requirement)
  - Trip DB: shard by trip_id (time-based or region-based)
  - Idempotency: duplicates possible (mobile retries); dedup via request ID
```

7.6 Design Netflix / Video Streaming

```
Requirements:
  - Stream video on any device
  - Adaptive bitrate (ABR) for varying network conditions
  - Content discovery (recommendations, search)
  - User profiles, watch history, continue watching
  - 260M+ subscribers, 2000+ devices

Architecture Components:

  Client Apps (2000+ devices):
    - Video player with ABR logic
    - UI rendering (customizable per device)
    - Local caching for thumbnails, metadata

  CDN (Open Connect):
    - Netflix's own CDN deployment: ISP-partnered appliances
    - Caches popular content at ISP edge
    - Custom hardware: optimized for video delivery
    - Handles 95%+ of traffic directly from edge

  Content Management:
    - Content Ingestion: process raw content from studios
    - Content Pipeline:
      1. Receive raw video (mezzanine files)
      2. QC and validation
      3. Transcode to 100+ formats
      4. Generate audio tracks (5.1, stereo, descriptive)
      5. Create subtitles
      6. Package for streaming

  Encoding Ladders:
    - 4K (HDR): 40 Mbps
    - 1080p: 5-8 Mbps
    - 720p: 2-4 Mbps
    - 480p: 1-1.5 Mbps
    - 360p: 500-800 Kbps
    - Per-title encoding: optimized ladder per movie
      - Action movies: high bitrate (fast motion)
      - Slow content: lower bitrate

  Recommendation Engine:
    - Collaborative filtering: "people who watched X also watched Y"
    - Content-based: similar genres, actors, directors
    - Deep learning: neural collaborative filtering
    - A/B testing: test algorithms on user subsets
    - Evidence: user behavior (what you watch, when, skip, rewatch)

  Chaos Engineering:
    - Simian Army: terminate instances in production
    - Chaos Kong: simulate AWS region failure
    - Fault Injection Testing (FIT): inject latency, errors
    - Purpose: verify system survives failures

Scaling:
  - Microservices: 500+ services (each with specific responsibility)
  - API Gateway: 50M+ requests/second at peak
  - Cassandra: user data, viewing history
  - EVCache (Memcached-based): caching layer
```

```
  - S3 + Open Connect: video storage + delivery
  - Kafka: event streaming (play, pause, search, rate)
  - Spark + Flink: real-time analytics
  - Zuul: edge service (routing, resilience, monitoring)
```

7.7 Design Web Crawler / Search Engine

```
Requirements:
  - Crawl billions of web pages
  - Download and store page content
  - Build inverted index for search
  - Freshness: recrawl updated pages
  - Politeness: respect robots.txt, rate limits

Architecture:

  URL Frontier:
    - Priority queue of URLs to crawl
    - Politeness: separate queues per host
    - Prioritization: PageRank, freshness, depth
    - Back queue: host-specific queue to respect Crawl-Delay
    - Front queue: selector queues (priority levels)

  Crawling Flow:
    1. URL Frontier → Downloader
    2. Downloader fetches page (HTTP GET)
    3. Check robots.txt (per URL, cached with TTL)
    4. Check if already downloaded (duplicate detection, fingerprint)
    5. Store raw HTML in Document Store (S3/blob)
    6. Extract links → add to URL Frontier
    7. Extract text → send for indexing
    8. Check for sitemap.xml (additional URLs)

  Duplicate Detection:
    - Simhash: near-duplicate detection
    - Fingerprint: MD5/SHA of normalized content
    - URL normalization: strip www, trailing slash, fragment, default port

  Indexing Pipeline:
    1. Text Extraction: parse HTML, strip tags
    2. Tokenization: split into words/tokens
    3. Normalization: lowercase, stem, remove stop words
    4. Inverted Index: term → list of (doc_id, position, metadata)
    5. Forward Index: doc_id → term positions (for snippets)
    6. Index segments: periodically merge into larger segments

  Search Query Flow:
    1. Parse query: tokenize, normalize
    2. Query expansion: synonyms, spell correction
    3. Retrieve posting lists from inverted index
    4. Score documents (TF-IDF, BM25, PageRank)
    5. Rank and return top K results
    6. Generate snippets from forward index

Scaling:
  - Downloaders: distributed, hundreds of machines
  - URL Frontier: distributed queues per host (partition by host hash)
  - Document Store: key-value (doc_id → raw content)
  - Index: sharded by term (or term range)
  - Search: replicate index across multiple servers
```

---

8. PHASE 6: LOW-LEVEL DESIGN & OOP

8.1 Object-Oriented Design Principles

```
SOLID Principles (Uncle Bob):

  S - Single Responsibility:
    A class should have only one reason to change
    Each class = one responsibility
    Example: ReportGenerator (generates) ≠ ReportPrinter (prints)

  O - Open/Closed:
    Open for extension, closed for modification
    Add new behavior without changing existing code
    Use: inheritance, composition, strategy pattern

  L - Liskov Substitution:
    Subclass should be substitutable for parent class
    If S is subtype of T, objects of T can be replaced with S
    Violation: Rectangle → Square (set_width changes height in Square)

  I - Interface Segregation:
    Clients shouldn't depend on interfaces they don't use
    Many specific interfaces > one general interface
    Example: Worker → Workable (work) + Eatable (eat) for robots

  D - Dependency Inversion:
    High-level modules shouldn't depend on low-level modules
    Both should depend on abstractions (interfaces)
    Depend on abstractions, not concretions

Additional Principles:
  DRY: Don't Repeat Yourself
  YAGNI: You Aren't Gonna Need It
  KISS: Keep It Simple, Stupid
  Composition over Inheritance
  Encapsulate what varies
  Program to an interface, not an implementation
  Favor composition over inheritance
  Law of Demeter (principle of least knowledge)
```

8.2 Design Patterns (with C++ Examples)

```
Creational Patterns:

  Singleton:
    class Singleton {
    public:
        static Singleton& getInstance() {
            static Singleton instance;
            return instance;
        }
        Singleton(const Singleton&) = delete;
        Singleton& operator=(const Singleton&) = delete;
    private:
        Singleton() = default;
    };
    - Thread-safe in C++11 (static initialization)
    - Use: logging, configuration, thread pool
    - Avoid: global state, testing difficulty
```

```
  Factory Method:
    class Product { public: virtual void use() = 0; };
    class ConcreteProductA : public Product {
    public: void use() override { std::cout << "Product A\n"; }
    };
    class Creator {
    public:
        virtual std::unique_ptr<Product> createProduct() = 0;
    };
    class ConcreteCreatorA : public Creator {
    public:
        std::unique_ptr<Product> createProduct() override {
            return std::make_unique<ConcreteProductA>();
        }
    };

  Builder:
    class Pizza {
    public:
        class Builder {
        public:
            Builder& setSize(int s) { size = s; return *this; }
            Builder& addTopping(std::string t) { toppings.push_back(t); return
            *this; }
            Pizza build() { return Pizza(size, toppings); }
        private:
            int size = 12;
            std::vector<std::string> toppings;
        };
    private:
        Pizza(int s, std::vector<std::string> t) : size(s), toppings(t) {}
        int size;
        std::vector<std::string> toppings;
    };
    // Usage: Pizza p =
    Pizza::Builder().setSize(16).addTopping("pepperoni").build();

Structural Patterns:

  Adapter:
    class Target {
    public: virtual void request() = 0;
    };
    class Adaptee {
    public: void specificRequest() { std::cout << "Specific request\n"; }
    };
    class Adapter : public Target {
        Adaptee* adaptee;
    public:
        Adapter(Adaptee* a) : adaptee(a) {}
        void request() override { adaptee->specificRequest(); }
    };

  Decorator:
    class Coffee {
    public: virtual double cost() = 0; virtual std::string description() = 0;
    };
    class SimpleCoffee : public Coffee {
    public:
```

```
        double cost() override { return 5.0; }
        std::string description() override { return "Coffee"; }
    };
    class MilkDecorator : public Coffee {
        std::unique_ptr<Coffee> coffee;
    public:
        MilkDecorator(std::unique_ptr<Coffee> c) : coffee(std::move(c)) {}
        double cost() override { return coffee->cost() + 1.5; }
        std::string description() override { return coffee->description() + " +
        Milk"; }
    };

  Facade:
    class ComputerFacade {
        CPU cpu;
        Memory memory;
        HardDrive hd;
    public:
        void start() {
            cpu.freeze();
            memory.load(0, hd.read(0, 1024));
            cpu.jump(0);
            cpu.execute();
        }
    };

Behavioral Patterns:

  Observer:
    class Observer {
    public: virtual void update(const std::string& msg) = 0;
    };
    class Subject {
        std::vector<Observer*> observers;
    public:
        void attach(Observer* o) { observers.push_back(o); }
        void notify(const std::string& msg) {
            for (auto o : observers) o->update(msg);
        }
    };

  Strategy:
    class SortingStrategy {
    public: virtual void sort(std::vector<int>& data) = 0;
    };
    class QuickSort : public SortingStrategy {
    public: void sort(std::vector<int>& data) override { /* quicksort */ }
    };
    class MergeSort : public SortingStrategy {
    public: void sort(std::vector<int>& data) override { /* mergesort */ }
    };
    class Sorter {
        std::unique_ptr<SortingStrategy> strategy;
    public:
        void setStrategy(std::unique_ptr<SortingStrategy> s) {
            strategy = std::move(s);
        }
        void applySort(std::vector<int>& data) { strategy->sort(data); }
    };
```

```
Command:
  class Command {
  public: virtual void execute() = 0; virtual void undo() = 0;
  };
  class TextEditor {
      std::string text;
  public:
      void insert(const std::string& s) { text += s; }
      void remove(int n) { text.erase(text.size() - n); }
  };
  class InsertCommand : public Command {
      TextEditor& editor;
      std::string text;
  public:
      InsertCommand(TextEditor& e, std::string t) : editor(e), text(t) {}
      void execute() override { editor.insert(text); }
      void undo() override { editor.remove(text.size()); }
  };

Iterator:
  template<typename T>
  class Iterator {
  public:
      virtual bool hasNext() = 0;
      virtual T next() = 0;
  };
  template<typename T>
  class Collection {
  public:
      virtual std::unique_ptr<Iterator<T>> createIterator() = 0;
  };

State:
  class VendingMachine;
  class State {
  public:
      virtual void insertCoin(VendingMachine& vm) = 0;
      virtual void dispense(VendingMachine& vm) = 0;
  };
  class NoCoinState : public State {
  public:
      void insertCoin(VendingMachine& vm) override;
      void dispense(VendingMachine& vm) override {
          std::cout << "Insert coin first\n";
      }
  };
  class HasCoinState : public State {
  public:
      void insertCoin(VendingMachine& vm) override {
          std::cout << "Coin already inserted\n";
      }
      void dispense(VendingMachine& vm) override;
  };
```

8.3 Common LLD Design Problems

```
Design a Parking Lot:
  Requirements:
    - Multiple floors, multiple spots per floor
    - Different spot types (compact, large, handicapped, electric)
    - Ticket generation on entry
```

```
    - Fee calculation on exit
    - Display available spots

  Key Classes:
    class ParkingLot {
        vector<Floor> floors;
        ParkingTicket issueTicket(Vehicle v);
        bool park(Vehicle v, ParkingSpot spot);
        double exit(ParkingTicket ticket);
    };
    class Floor {
        int floorNumber;
        unordered_map<SpotType, vector<ParkingSpot>> spots;
        int getAvailableCount(SpotType type);
    };
    class ParkingSpot {
        int id; bool isOccupied; SpotType type;
        bool occupy(); bool vacate();
    };
    class Vehicle {
        string licensePlate; VehicleType type;
    };
    class ParkingTicket {
        string id; time_t entryTime; time_t exitTime;
        int spotId; double fee;
    };
    class FeeCalculator {
        virtual double calculate(time_t duration, SpotType type) = 0;
    };

Design a Library Management System:
  Key Classes:
    class Library {
        Catalog catalog;
        vector<Member> members;
        vector<Librarian> librarians;
    };
    class Catalog {
        unordered_map<string, vector<BookItem>> booksByTitle;
        unordered_map<string, vector<BookItem>> booksByAuthor;
        unordered_map<string, vector<BookItem>> booksBySubject;
        vector<BookItem> search(string query, SearchType type);
    };
    class BookItem {
        string barcode; Book book; bool isAvailable;
        time_t dueDate; Member* borrowedBy;
        void checkout(Member* m, time_t due);
        void returnBook();
    };
    class Member {
        string memberId; string name; int maxBooks;
        vector<BookItem> checkedOut;
        vector<Fine> fines;
        bool checkout(BookItem* book);
        void returnBook(BookItem* book);
    };
    class Fine {
        double amount; BookItem* book; time_t date;
    };
```

```
Design a Chess Game:
  Key Classes:
    enum Color { WHITE, BLACK };
    enum PieceType { KING, QUEEN, ROOK, BISHOP, KNIGHT, PAWN };

    class Piece {
    protected:
        Color color; Position pos; PieceType type;
    public:
        virtual vector<Position> getValidMoves(Board& board) = 0;
        bool makeMove(Position dest, Board& board);
    };
    class Board {
        Piece* squares[8][8];
    public:
        bool isCheck(Color kingColor);
        bool isCheckmate(Color kingColor);
        bool isStalemate(Color kingColor);
    };
    class Game {
        Board board; vector<Player> players;
        Color currentTurn; GameStatus status;
        vector<Move> moveHistory;
        bool makeMove(Player p, Position from, Position to);
    };
    class Player {
        string name; Color color;
    };

Design an ATM Machine:
  Key Classes:
    class ATM {
        CashDispenser dispenser;
        CardReader cardReader;
        Screen screen;
        Keypad keypad;
        Printer printer;
        BankService bank;
        bool authenticateCard(Card c, int pin);
        Transaction executeTransaction(Transaction t);
    };
    class BankService {
        Account getAccount(string cardNumber);
        bool authenticatePin(string cardNumber, int pin);
        bool withdraw(string account, double amount);
        double getBalance(string account);
    };
    class Transaction {
        string transactionId; TransactionType type;
        double amount; time_t timestamp;
        virtual bool execute() = 0;
    };
    class Withdrawal : public Transaction {
        Account account; double amount;
    };
    class Deposit : public Transaction {
        Account account; double amount; Envelope envelope;
    };
```

9. PHASE 7: INTERVIEW PREPARATION

9.1 HLD Interview Framework

```
The 4-Step Approach (40-45 minutes):

Step 1: Requirements (5 min)
  - Functional requirements: What should the system do?
  - Non-functional requirements: Scale, latency, consistency, availability
  - Constraints: Budget, timeline, team size

  Example questions to ask:
    "How many DAU?" "What are the write/read ratios?"
    "Do we need real-time?" "What's the expected latency?"

Step 2: High-Level Design (10 min)
  - Draw main components: clients, load balancers, servers, DB, cache
  - Define APIs (REST/gRPC endpoints)
  - Define data models (DB schema)
  - Show data flow (read path, write path)

Step 3: Deep Dive (20 min)
  - Scale: How to handle 10x, 100x more traffic?
  - Performance: Caching strategies, CDN, read replicas
  - Consistency: Strong vs eventual, tradeoffs
  - Availability: Replication, failover, circuit breakers
  - Data partitioning: Sharding strategies, consistent hashing
  - Specific challenges based on the problem

Step 4: Summary & Tradeoffs (5 min)
  - Recap your design
  - Tradeoffs you made (why?)
  - Alternative approaches considered
  - What would you improve with more time?
  - Bottlenecks and failure modes

Common Mistakes to Avoid:
  - Jumping to details without understanding requirements
  - Not mentioning tradeoffs
  - Being too silent (verbalize your thinking)
  - Forgetting about monitoring, logging, deployment
  - Over-engineering for current scale
  - Single point of failure in design
```

9.2 LLD Interview Framework

```
The 3-Step Approach (35-40 minutes):

Step 1: Clarify Requirements (5 min)
  - What entities exist? What are their relationships?
  - What operations are needed? (CRUD, specific actions)
  - Any constraints? (thread safety, persistence, time limits)

Step 2: Class Design (15 min)
  - Identify core classes and their relationships
  - Define interfaces (abstract classes)
  - Show inheritance hierarchy
  - Discuss design patterns used
  - Write skeleton code on whiteboard/editor

Step 3: Implementation (15 min)
  - Implement key methods
```

```
  - Show how objects interact
  - Handle edge cases
  - Discuss thread safety if needed
  - Time and space complexity analysis

Common LLD Problems:
  - Design a parking lot
  - Design a library management system
  - Design a chess game
  - Design a vending machine
  - Design an elevator system
  - Design a hotel booking system
  - Design an ATM
  - Design a restaurant ordering system
  - Design a task management system
  - Design a logging framework
```

9.3 30+ Common System Design Questions

```
High-Level Design Questions:
  1. Design URL shortener (bit.ly)
  2. Design WhatsApp / messaging app
  3. Design Twitter / social feed
  4. Design YouTube / video streaming
  5. Design Uber / ride-hailing
  6. Design Netflix
  7. Design web crawler / search engine
  8. Design Dropbox / file storage
  9. Design Instagram / photo sharing
  10. Design YouTube / video platform
  11. Design Amazon / e-commerce
  12. Design Airbnb / booking platform
  13. Design Zoom / video conferencing
  14. Design Google Maps / navigation
  15. Design Ticketmaster / booking
  16. Design collaborative editor (Google Docs)
  17. Design Pastebin / code sharing
  18. Design notification system
  19. Design rate limiter
  20. Design CDN
  21. Design distributed cache
  22. Design distributed key-value store
  23. Design distributed messaging queue
  24. Design parking lot (also LLD)
  25. Design recommendation system
  26. Design payment system
  27. Design chat system
  28. Design API rate limiter
  29. Design proximity service (Yelp)
  30. Design distributed logger

Low-Level Design Questions:
  1. Design a parking lot
  2. Design a library management system
  3. Design a chess game
  4. Design a vending machine
  5. Design an elevator system
  6. Design an ATM machine
  7. Design a hotel booking system
  8. Design a ticket booking system
  9. Design a restaurant management system
```

```
10. Design a task management system (Trello)
11. Design a logging framework
12. Design an e-commerce cart system
13. Design a coffee machine
14. Design a snake and ladder game
15. Design a traffic light control system
```

9.4 Key Calculations for Interviews

```
Estimating QPS (Queries Per Second):
  DAU × actions per user per day = total daily actions
  Total / 86400 (seconds in day) = average QPS
  Average × peak factor (3–10x) = peak QPS

  Example: Twitter 500M DAU, 50 tweets/user/day
    Total tweets: 500M × 50 = 25B/day
    Average QPS: 25B / 86400 ≈ 290K/s
    Peak QPS: 290K × 3 ≈ 870K/s

Storage Estimation:
  DAU × data per user per day = daily storage
  Daily × retention days (or years × 365)

  Example: Twitter 500M tweets/day, 1KB each
    Daily: 500M × 1KB = 500 GB
    Yearly: 500GB × 365 ≈ 182 TB
    5 years: ~1 PB

Bandwidth Estimation:
  Data per request × QPS = bandwidth
  Write: inbound bandwidth
  Read: outbound bandwidth

  Example: 870K write QPS × 1KB = 870 MB/s
    ~7 Gbps inbound (at peak)

Caching Estimation:
  Cache hit ratio: 95%+ is typical
  Cache size: percentage of hot data (10–20% of total)
  Memory per server × number of servers = total cache capacity

  Example: 1M daily active items, 100KB each
    Hot set (20%): 200K × 100KB = 20 GB
    Redis cluster: 10 × 32GB = 320GB (headroom)

Server Count Estimation:
  Total QPS / QPS per server = servers needed

  Example: 1M QPS, modern server handles 50K QPS
    Servers: 1M / 50K = 20 servers
    With redundancy (2x): 40 servers
```

---

10. TOOLS & TECHNOLOGIES TABLE

10.1 Backend & API

Tool	Type	Use Case	Learning Curve
NGINX	Web server / LB	Reverse proxy, caching, LB	Medium

Tool	Type	Use Case	Learning Curve
HAProxy	Load balancer	TCP/HTTP load balancing	Medium
Envoy	Service mesh proxy	Microservice communication	High
FastAPI	Python web framework	REST APIs	Low
Spring Boot	Java framework	Enterprise microservices	Medium
Express.js	Node.js framework	Lightweight APIs	Low
gRPC	RPC framework	Service-to-service	Medium
GraphQL	Query language	Flexible APIs	Medium

10.2 Databases

Tool	Type	Use Case	Consistency
PostgreSQL	Relational	Transactions, complex queries	Strong
MySQL	Relational	Web apps, LAMP stack	Strong
MongoDB	Document	Flexible schema	Eventual/Strong
Cassandra	Wide-column	Time-series, write-heavy	Eventual
Redis	Key-value	Cache, real-time	Eventual/Strong
Elasticsearch	Search	Full-text search	Eventual
DynamoDB	Key-value	Serverless apps	Eventual/Strong
CockroachDB	NewSQL	Distributed SQL	Strong
Spanner	NewSQL	Global consistency	Strong

10.3 Message Queues & Streaming

Tool	Type	Use Case	Throughput
Kafka	Distributed log	Event streaming	1M+/s
RabbitMQ	Message broker	Task queues	50K/s
AWS SQS	Managed queue	Serverless apps	Unlimited
AWS SNS	Pub/sub	Notifications	Unlimited
Pulsar	Streaming	Multi-tenant	1M+/s

10.4 Monitoring & Observability

Tool	Type	Use Case	Cost
Prometheus	Metrics	Time-series monitoring	Free
Grafana	Visualization	Dashboards	Free
ELK Stack	Logging	Log aggregation	Free/Paid
Datadog	Full observability	APM, logs, metrics	Paid
New Relic	Full observability	APM	Paid
OpenTelemetry	Instrumentation	Standard	Free
Jaeger	Tracing	Distributed tracing	Free

Tool	Type	Use Case	Cost

10.5 Cloud & Infrastructure

Tool	Type	Use Case	Popularity
AWS	Cloud provider	General cloud	Highest
GCP	Cloud provider	Big data, ML	Growing
Azure	Cloud provider	Enterprise	High
Terraform	IaC	Infrastructure provisioning	High
Kubernetes	Orchestration	Container management	Highest
Docker	Containerization	Application packaging	Highest
Istio	Service mesh	Microservice management	Growing

---

11. RESOURCES & BOOKS

11.1 Essential Books

```
System Design:
  - "Designing Data-Intensive Applications" by Martin Kleppmann ★★
    THE book for system design. Covers distributed systems deeply.
    Partitions, replication, transactions, consensus, streams.

  - "System Design Interview" by Alex Xu (Vol 1 & 2) ★
    Interview-focused, case studies. Good for interview prep.
    Practical examples with diagrams.

  - "The System Design Interview" by Lewis Lin
    Step-by-step interview framework.

  - "Building Microservices" by Sam Newman
    Microservices patterns, deployment, testing.

  - "Fundamentals of Software Architecture" by Richards & Ford
    Architecture styles, patterns, decision making.

Distributed Systems:
  - "Distributed Systems" by Maarten van Steen (free online)
  - "Distributed Computing" by Attiya & Welch
  - "Distributed Algorithms" by Lynch (advanced)

Low-Level Design:
  - "Head First Design Patterns" by Freeman & Robson ★
    Best intro to design patterns.

  - "Design Patterns" by Gang of Four (GoF)
    The classic reference.

  - "Clean Code" by Robert Martin
    Code organization, naming, testing.

  - "Refactoring" by Fowler
    Improving existing designs.
```

```
Specific Topics:
  - "Database Internals" by Petrov
  - "Streaming Systems" by Akidau et al.
  - "Kafka: The Definitive Guide" by Narkhede et al.
  - "High Performance MySQL" by Schwartz et al.
  - "Redis in Action" by Carlson
```

11.2 Online Courses & Resources

```
Courses:
  - Grokking the System Design Interview (Educative) ★
    Essential interview prep. Covers 20+ designs.

  - Grokking Advanced System Design (Educative)
    Distributed system patterns, deeper dives.

  - System Design Interview (AlgoExpert)
    Video explanations, diagrams.

  - Designing Data-Intensive Applications (O'Reilly)
    Book club, deeper study.

YouTube Channels:
  - Gaurav Sen (System Design) ★
    Excellent explanations, Indian accent.

  - Tech Dummies (Design case studies)
    Detailed walkthroughs.

  - Hussein Nasser (System design, databases)
    Deep technical content.

  - ByteByteGo (Alex Xu)
    Animated explanations of popular designs.

  - System Design Interview (Jordan has no life)
    FAANG-specific prep.

Blogs & Articles:
  - High Scalability (highscalability.com) ★
    Real-world architectures from big companies.

  - The Architecture Notes (Mahbub.com)
    Case studies and patterns.

  - Martin Fowler's Blog
    Design patterns, refactoring.

  - Joel on Software
    Classic software engineering essays.

Practice Platforms:
  - Pramp: Mock system design interviews (peer-to-peer)
  - Interviewing.io: Anonymous mock interviews
  - ByteByteGo: Mock interview practice
  - Exponent: System design + behavioral
```

11.3 Study Timeline

```
Weeks 1-2: CS Refresher + Fundamentals
  - OS: processes, threading, memory, sync
```

```
  - Networking: TCP/UDP, HTTP/2/3, DNS
  - CAP, PACELC, consistency models
  - Read: DDIA Chapters 1-4

Weeks 3-4: Storage & Databases
  - SQL vs NoSQL tradeoffs
  - Indexing, replication, sharding
  - Caching strategies
  - Read: DDIA Chapters 5-8

Weeks 5-6: Communication + APIs
  - REST API design
  - gRPC vs REST vs GraphQL
  - Message queues (Kafka, RabbitMQ)
  - Rate limiting, proxies

Weeks 7-8: Distributed Systems + Patterns
  - Consensus (Raft, Paxos)
  - Microservices patterns
  - Monitoring, observability
  - Read: DDIA Chapters 9-11

Weeks 9-12: High-Level Design Case Studies
  - Practice 15+ HLD problems
  - URL shortener, WhatsApp, Twitter
  - YouTube, Uber, Netflix
  - Mock interviews (2-3 per week)

Weeks 13-14: Low-Level Design + OOP
  - SOLID + Design Patterns
  - Practice 10+ LLD problems
  - Code implementations in C++
  - Mock LLD interviews

Week 15-16: Final Review + Mock Interviews
  - Review all key concepts
  - Stress test weak areas
  - 5+ full mock interviews
  - Practice estimation calculations
```

FINAL WORDS

System Design is a conversation, not a presentation. Engage with your interviewer, ask clarifying questions, discuss tradeoffs. They want to see how you think, not just what you know.

Learn the fundamentals deeply. CAP theorem, consistency models, and how databases work internally will serve you for every design problem. Don't memorize solutions — understand principles.

Practice drawing. A clear diagram is worth 1000 words. Practice on whiteboards, digital tools (Excalidraw), or paper. Components should be labeled, arrows should show data flow.

Always discuss tradeoffs. Every design decision has pros and cons. Mentioning both shows depth. "I'd use Cassandra here for write scalability, but we'd sacrifice strong consistency and complex querying."

Scale progressively. Start with a simple design that works for small scale, then discuss how to scale it. Don't start with 1000 microservices — show you can build incrementally.

Know your numbers. Requests per second, storage requirements, bandwidth, cache sizes, number of servers. These calculations show you can make data-driven decisions.

The best design is the one that works for the problem. Don’t over-engineer. Sometimes a monolith with a single database is the right answer.